ARCHITECTURE DECISION DOCUMENT

RAG Architecture Options for Structured, Verdict-Heavy Data

MEAN-stack integration patterns, multi-tenancy models, and three competing storage & retrieval tracks for a Retrieval-Augmented Generation system over checklist-style inspection reports (Section → Question → Answer hierarchy) alongside a free-form document library.

Covers: MEAN integration patterns · RBAC & multi-tenant models · Postgres-native vs. dedicated vector DB vs. MongoDB Atlas · packaged options & recommendation

1	Fitting RAG into a MEAN Stack	Pure Node.js vs. Node + Python microservice
2	Per-User Pipelines	RBAC, tenant isolation, and the hybrid model
3	Storage & Retrieval Tracks	Postgres-native, dedicated vector DB, MongoDB Atlas
4	Three Packaged Options	Complete, buildable configurations
5	Recommendation	The strongest all-round bet, and why
6	Open Questions	What to settle before committing

1 Fitting a RAG System into a MEAN Stack

"MEAN" commits the project to MongoDB, Express, Angular, and Node — but the RAG tooling ecosystem is still majority-Python. Two integration patterns are viable, and the choice affects the storage track decision in Section 3.

1.1 Pattern A — Pure Node.js/JS, No Second Runtime

Angular → Express → a JS orchestration layer (LangChain.js or hand-rolled) → a JS driver for whichever store is chosen → an LLM API. Since embeddings are almost always called via API (OpenAI, Cohere, Voyage) rather than run locally, nothing about hybrid search, structured retrieval, or relational/metadata querying strictly *requires* Python.

Pros

- One language across the whole stack
- Existing Angular/Node engineers can read and own the retrieval code
- Fewer services to deploy

Cons

- JS's document-parsing ecosystem is thinner than Python's (complex spreadsheets, OCR, layout-aware PDFs)
- Multimodal retrieval (vision-language models, video keyframing, transcription) is overwhelmingly Python-first

1.2 Pattern B — Node App Layer + Python RAG Microservice(s)

Node/Express stays the system of record for the application itself — frontend, auth, RBAC, session/audit data — and fronts one or more Python (FastAPI) services that own parsing, chunking, embedding, hybrid/graph retrieval, multimodal processing, and grounded generation.

Pros

- Right tool for each job
- Ingestion (slow, batch) scales independently from retrieval (interactive, latency-sensitive)
- Python owns the mature RAG/vision/video libraries

Cons

- Two runtimes to operate
- An internal API boundary to design and secure (REST/gRPC, or a queue like BullMQ/Redis for async ingestion jobs)

Recommendation: Start with Pattern A for a text-only first build — it's the fastest path to a working demo, since structured and relational retrieval are entirely text-based with API-driven embeddings and JS isn't a handicap there. Introduce the Python microservice boundary once multimodal retrieval (images/video) is in scope, since that's where the ecosystem gap actually bites.

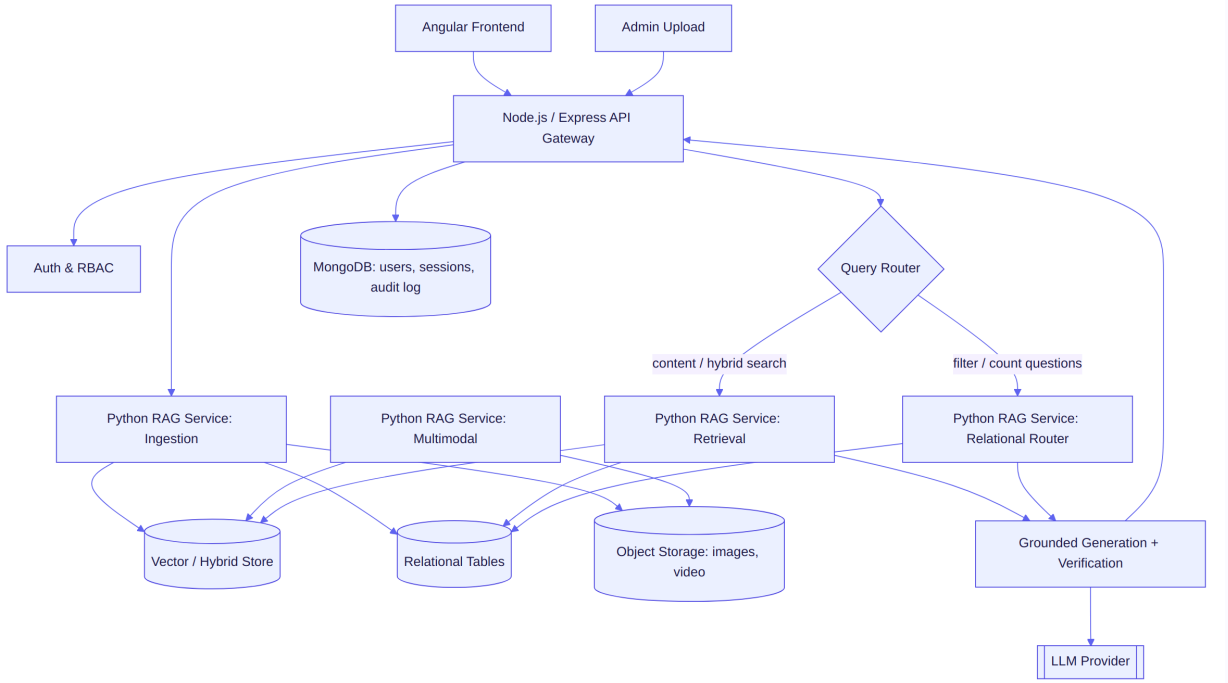


Figure 1.1 — Combined MEAN + Python RAG microservice architecture (Pattern B, full scope)

2 Per-User Pipelines: Three Models and Their Trade-offs

"Per-user" can mean two different problems, so it's worth separating them.

Model 1 — Role-Based Access Control (RBAC) Inside a Shared Store

One deployment, one corpus, but a given user only sees the slice their role/site/vendor entitles them to (e.g. a field inspector sees only their site's reports; an admin sees everything). Every chunk/row is tagged with access metadata (`site, inspector, vendor, PO, verdict`, etc.) at ingestion time.

Implementation: enforce the filter at the database layer — Postgres Row-Level Security keyed to JWT claims, or Mongo query filters injected server-side — never only in application code, since a missed `WHERE` clause in app code is a silent data leak.

Pros

- Simplest to build and operate
- One index to maintain
- Matches "different users, one org"

Cons

- Doesn't give a hard boundary for genuinely separate organizations with separate contracts/SLAs

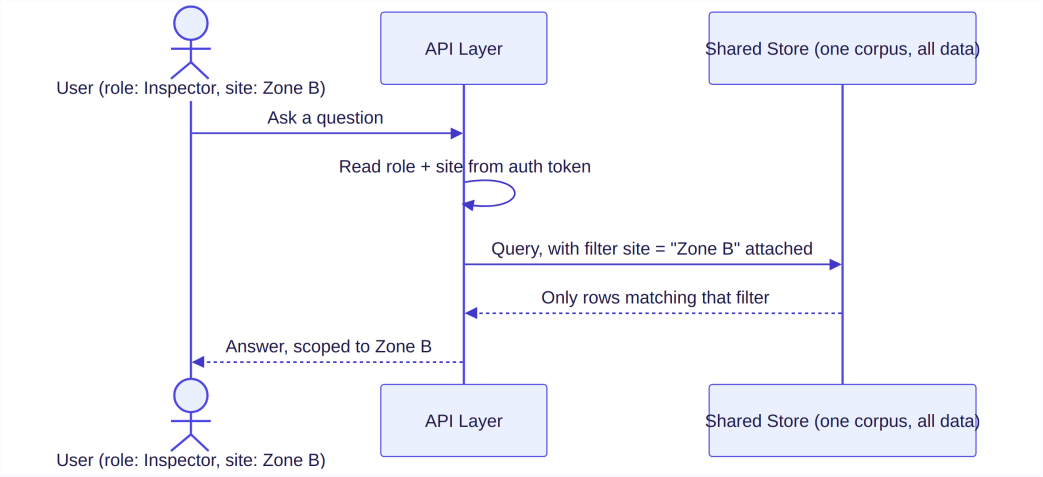


Figure 2.1 — The filter travels with the query itself; the database never holds data the user isn't entitled to see unfiltered.

Model 2 — Isolated Pipelines per Tenant

If the system is ever sold to multiple separate organizations (not just internal users of one org), each gets its own ingestion + storage: schema-per-tenant, database-per-tenant, or a vector-store-native tenant partition (several engines have this built in — see Section 3).

Pros

Cons

- Hard isolation boundary — easiest to defend in a security audit or contract
- Independent scaling/backup per tenant

- N pipelines to run and monitor
- Cost scales with tenant count
- Cross-tenant analytics become hard *by design*

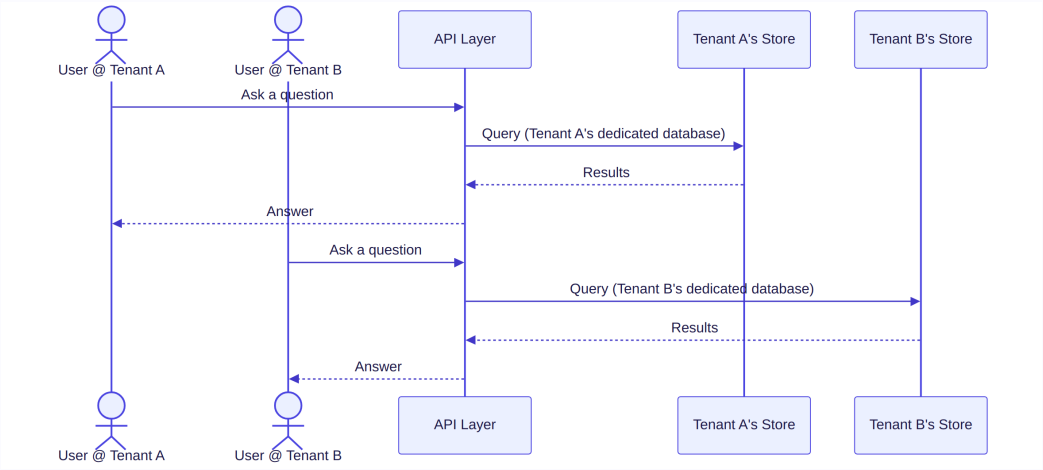


Figure 2.2 — There is no shared store to filter; Tenant A's query physically cannot reach Tenant B's database.

Model 3 — Hybrid: Tenant Isolation + Role Filtering Inside Each Tenant

Model 2's isolation, with Model 1's filtering applied inside each tenant's slice. This is generally the eventual production target once both true multi-org isolation *and* role-based access within each org are required — along with the rest of production hardening (rate limiting, token metering, ACL trimming) that a first build typically defers.

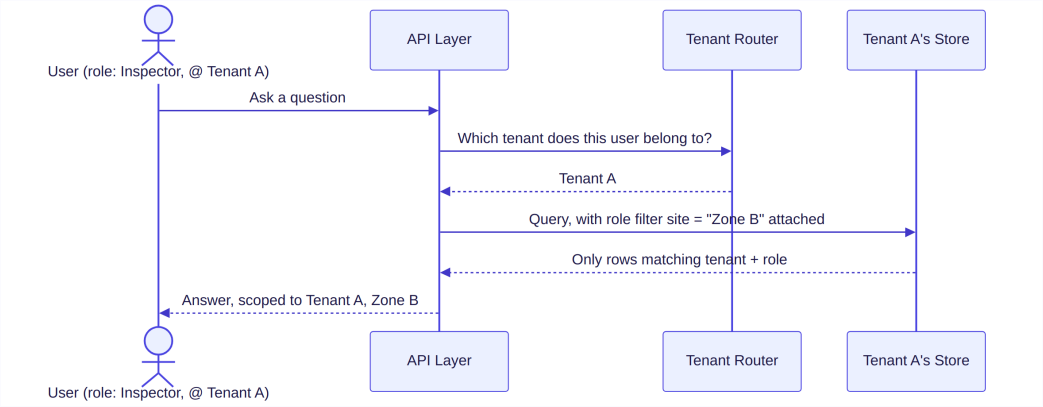


Figure 2.3 — Two checks stack: which tenant's database to query, then which rows inside it the user's role allows.

Recommendation: Build Model 1 first — it's the cheapest path to a working system and covers the common case of "different users, different visibility, one organization." But add `tenant_id` as a first-class field on every chunk/row **from day one**, even if there's only one tenant initially. That's the difference between Model 3 being a config change later versus a full re-ingestion project later.

3 Storage & Retrieval Track: Three Competing Options

For data that is rigidly structured and verdict-heavy — where exactness, structure, and clean citations matter more than raw search scale — three architectures are genuinely comparable.

Track A — Postgres-native (pgvector/pgContext + pgGraph + SQL)

One Postgres instance handles everything: pgContext (hybrid dense+keyword+metadata search, single engine) with pgvector + full-text kept live as a fallback; pgGraph for hierarchical/relationship traversal (e.g. a `Report → Section → Question` tree with shared Vendor/Spec/Inspector nodes); plain SQL for exact filter/count questions.

pgContext and pgGraph are real, open-source (Apache-2.0) Postgres extensions built by the same small team. That's not a knock on the design — combining hybrid search, graph traversal, and SQL in one engine is architecturally elegant — but pgGraph's own documentation currently marks it **alpha**, explicitly recommending Docker/dev use rather than a production or shared cluster, and pgContext is younger still. That's a real maturity risk to weigh, independent of the design's merits.

Pros

- One database to run, back up, and secure
- ACLs/RLS apply uniformly across vectors, text, and graph
- No sync problem between systems
- Cheapest to operate at moderate data volumes (thousands, not billions, of rows)

Cons

- Implementation risk sits on two sub-1.0 components from a small team
- Smaller support surface than the alternatives
- Graph performance at scale is less proven — though a shallow 3–4 level hierarchy is unlikely to stress it

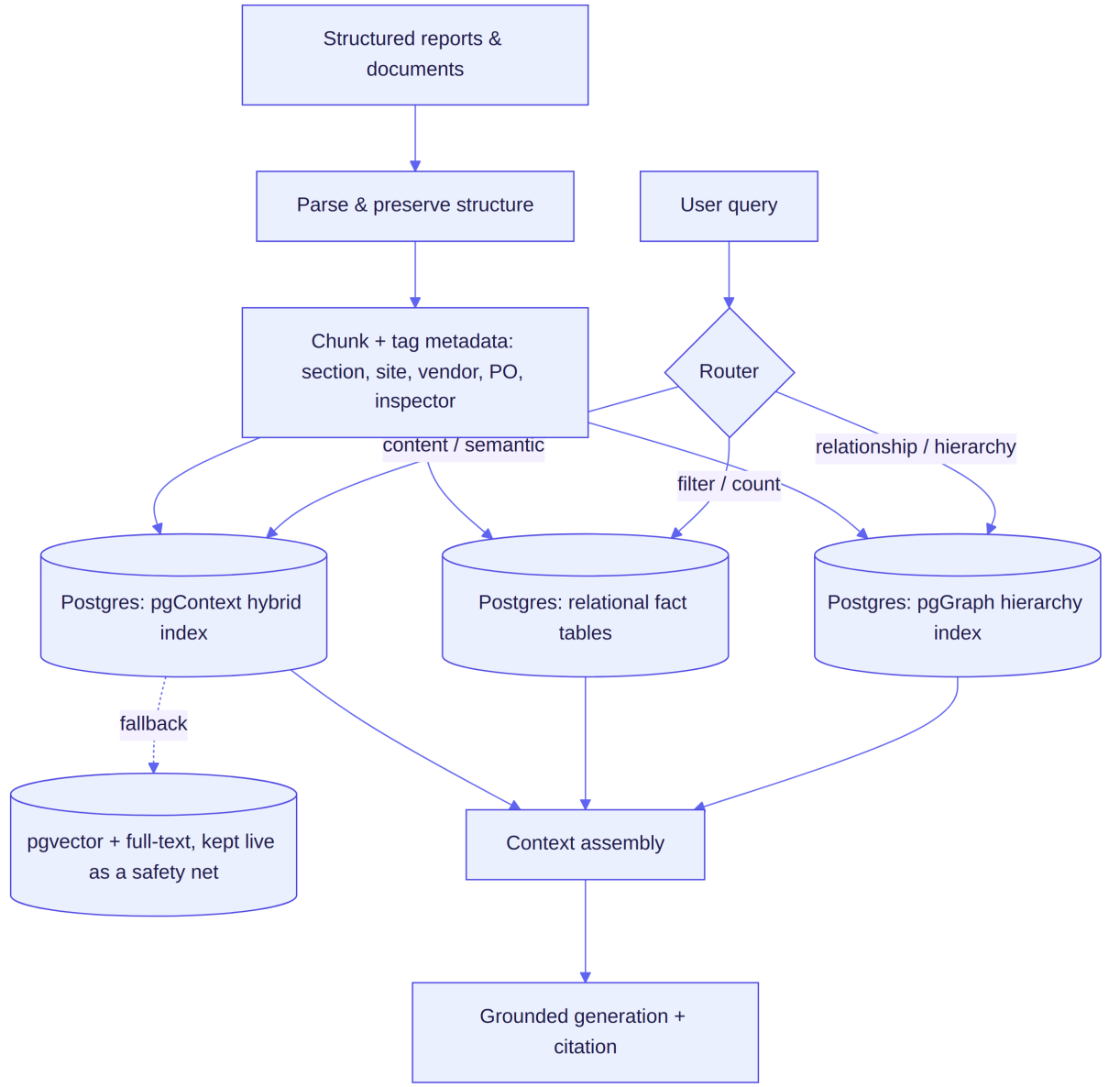


Figure 3.1 — Track A: one Postgres instance for hybrid search, relational tables, and graph hierarchy.

Track B — Dedicated Open-Source Vector DB + Postgres for Relational

Offload vector/hybrid search to a mature, purpose-built engine; keep Postgres for relational fact tables and the SQL router. At moderate scale, a shallow hierarchy (3–4 levels) can be handled with plain recursive CTEs in Postgres without needing a dedicated graph engine at all.

Engine	Best At	Trade-off	Fit for Verdict/Metadata-Heavy Data
Qdrant	Filtered search — Rust, single-binary, lowest self-hosted latency, tenant-keyed payload filters	Younger ecosystem than Milvus/Weaviate	**Strong fit** — queries are metadata-heavy (site, vendor, PO, date), exactly what payload-filtered HNSW is built for
Weaviate	Native hybrid search (BM25+vector), built-in embedding modules, strongest "batteries-included" multi-tenancy	GraphQL API learning curve, heavier (JVM-based) runtime	Good if less glue code and real multi-tenancy matter more than raw latency
Milvus	Billion-scale, GPU-accelerated indexing, Kubernetes-native, database-level tenant isolation	Real operational weight — generally only worth it once you're already managing Kubernetes	**Overkill** below tens/hundreds of millions of vectors
Chroma	Zero-config local prototyping	Not a production self-hosted store — no HA, no real multi-tenancy	Fine for a throwaway spike, not for a production build

Pros

- De-risks away from pre-1.0 dependencies
- Strong filter performance for exactly this query pattern
- Still cheap to operate at moderate scale

Cons

- A second database alongside Postgres
- Sync between the two needs to be designed (usually solved by writing both in the same ingestion transaction/job)

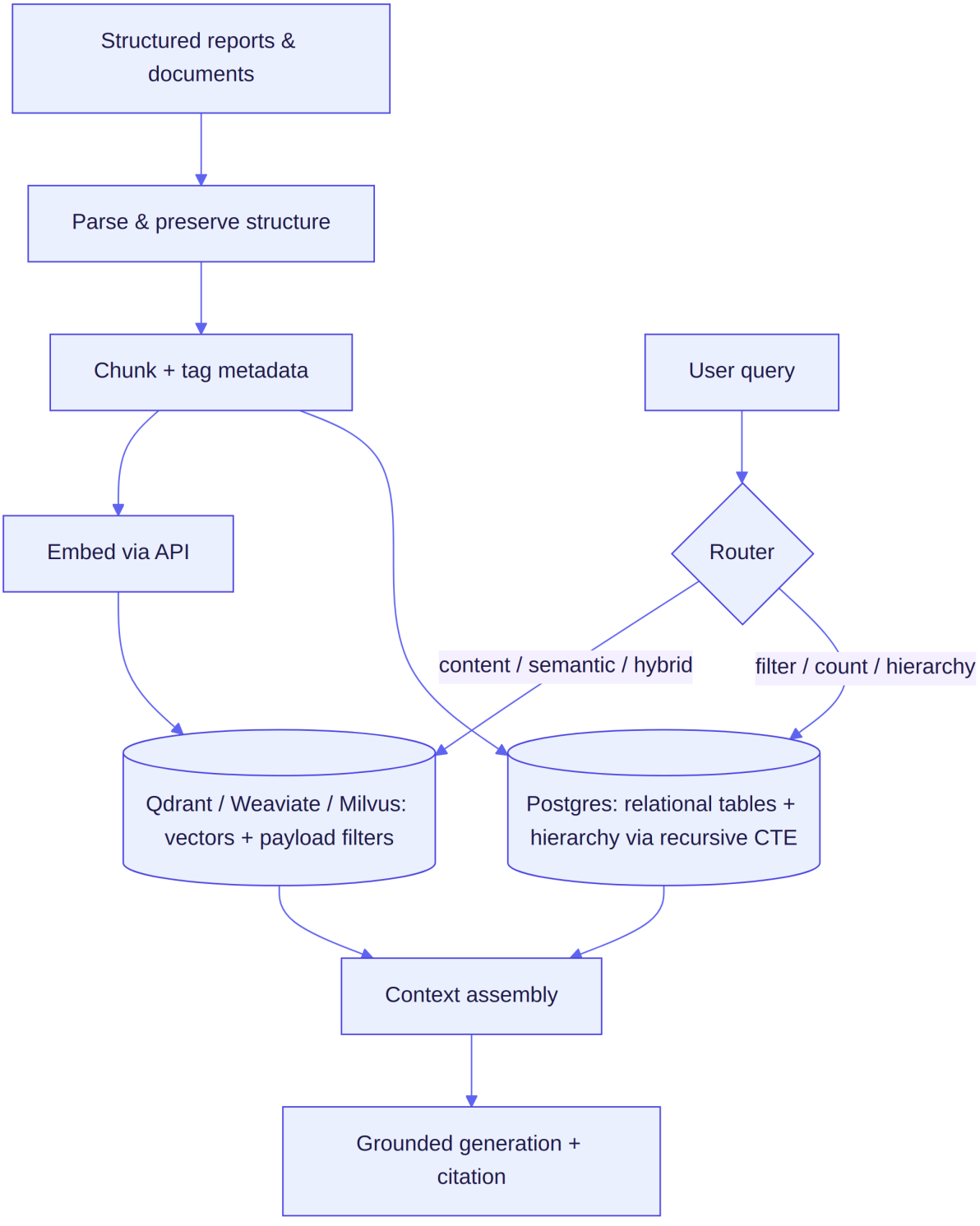


Figure 3.2 — Track B: purpose-built vector engine for search, Postgres for relational + hierarchy.

Track C — MongoDB Atlas Vector Search (Native Fit for a MEAN Stack)

Since "M" in MEAN is MongoDB, this deserves a place at the table: Atlas supports vector search, native hybrid search (`$search` + `$vectorSearch` fused via `$rankFusion` / `$scoreFusion`), and pre-filtering on document fields at index time — including a documented pattern of denormalizing a `tenant_id` / `workspaceId` field onto every chunk so multi-tenant filtering happens *inside* the HNSW traversal rather than as a slower downstream `$match` . An existing `mongodb-rag` npm package also scaffolds an Express + Atlas RAG app directly.

Pros

- Zero new database to introduce — vectors, metadata, and existing app data live in one cluster under one security model
- Genuinely the lowest-friction option for a team already on MEAN
- Native hybrid search is a checkbox, not a build

Cons

- No native graph traversal (nested documents or `$graphLookup` — far weaker for deep traversals, though a shallow checklist tree may not need much)
- Exact count/filter relies on the aggregation pipeline — serviceable but less naturally auditable
- MongoDB's own multi-tenancy guidance favors a single collection + `tenant_id` , mapping well to Model 1 but less well to Model 2/3

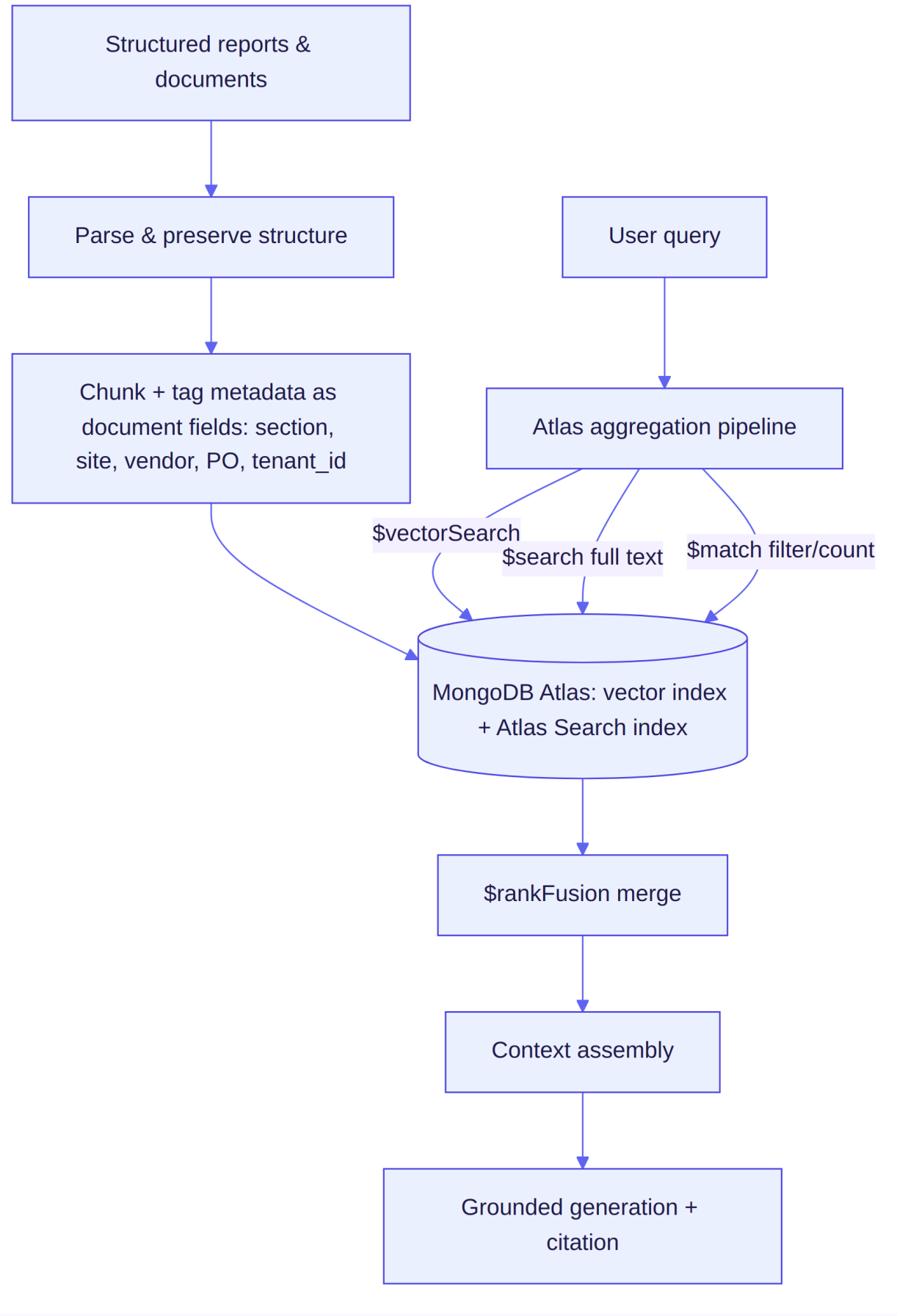


Figure 3.3 — Track C: everything — app data, vectors, and search indexes — inside one MongoDB Atlas cluster.

Track Comparison at a Glance

	Track A: Postgres-native	Track B: Dedicated Vector DB + Postgres	Track C: MongoDB Atlas
New infra beyond MEAN	Yes — Postgres	Yes — Postgres + vector engine	**No**
Hybrid search	Native (pgContext)	Native (Qdrant/Weaviate)	Native ($rankFusion)
Graph/hierarchy	Native (pgGraph)	Postgres recursive CTE (fine at shallow depth)	Weak ($graphLookup, nested docs)
Exact filter/count	Native SQL	Native SQL	Aggregation pipeline (workable, less auditable)
Maturity risk	**Higher** — two sub-1.0 extensions	Low — all options battle-tested	Low — Atlas Vector Search is GA
Multi-tenancy model	RLS (Model 1) or schema/db split (Model 2/3)	RLS/payload filter (Model 1) or native partitions (Weaviate/Milvus)	Single-collection + tenant_id (Model 1); hard isolation not MongoDB's recommended pattern
Ops overhead	One database	Two databases	One database (already in-stack)

4 Three Packaged Options

Combining Sections 1–3 into complete, buildable configurations:

	Option 1 — Ship Fast	Option 2 — Best-of-Breed, Still Lean	Option 3 — MEAN-Native, No New Infra
MEAN pattern	Pattern A (pure Node.js)	Pattern B (Python microservice)	Pattern A or B — storage choice doesn't force this one
Storage track	Track A (Postgres pgContext/pgGraph, pgvector fallback live)	Track B (Qdrant + Postgres relational)	Track C (MongoDB Atlas)
Per-user model	Model 1 (RBAC), tenant_id reserved	Model 1 (RBAC), tenant_id reserved	Model 1 (RBAC via tenant_id field — MongoDB's own recommended pattern)
Best for	Fastest path to a working demo	Lower dependency risk, strong filter performance, still cheap at moderate scale	Minimum new infrastructure; staying entirely inside MongoDB
Main risk	Betting retrieval quality on alpha-stage extensions	Two databases to keep in sync	Graph/hierarchy modeling is the weakest link; exact-count auditability is softer than SQL

The storage track is realistically swappable later, since the ingestion pipeline (parse → chunk → tag metadata) is shared across all three.

5 Recommendation

Given data that is rigidly structured and verdict-heavy — where exactness and clean citations matter more than raw search scale:

Primary pick

Option 2 — Track B: Qdrant + Postgres

The strongest all-round bet if introducing Postgres alongside MongoDB is acceptable: it keeps the exactness Track A targets, without staking retrieval quality on two sub-1.0 extensions, and Qdrant's filter-first design matches this query pattern about as well as pgContext's does.

If MongoDB is a hard constraint

Option 3 — Track C: MongoDB Atlas

The pragmatic choice if staying entirely inside MongoDB is required — graph traversal is lost, but a shallow checklist hierarchy may not need it, and it can always be added later via a small side-service if hierarchy queries turn out to need deeper relationship reasoning than expected.

Track A is still worth piloting in parallel, not instead — it's the most architecturally elegant of the three (one engine, everything), and keeping pgvector + full-text live as a fallback is the right way to de-risk it. Running it as a side-by-side spike alongside whichever primary track is chosen, rather than betting on it outright, answers the maturity question with real data instead of vendor claims.

6 Questions Worth Answering Before Committing

1. Is MongoDB a hard commitment for all data?

Or is polyglot persistence (MongoDB for app data, a second store for retrieval) acceptable? This single answer eliminates two of the three tracks in Section 3.

2. Will the system ever serve more than one organization?

Or is "per user" entirely about roles within one org? Decides between Model 1 alone versus reserving for Model 3.

3. What's the tolerance for pre-1.0 dependencies?

In a system meant to reach production — decides how much weight Track A's maturity risk should carry versus Track B/C.

4. Expected data volume and growth over 12–24 months?

Directly decides whether Milvus's scale advantage in Track B ever becomes relevant, which at moderate report-corpus volumes it currently does not.